Secure User Login and Management System

Computer and Information Security | Third-Year Software Engineering | Project Implementation Report

1. Introduction and objectives

This project demonstrates a local web application that identifies users, authenticates them with a password and a time-based one-time code, and authorizes actions based on assigned roles. The aim is to balance straightforward user interaction with foundational security controls.

2. Technologies and architecture

The frontend uses HTML5, CSS3 and Bootstrap 5 templates; Python Flask handles HTTP requests and sessions; SQLite stores accounts and sample records locally. The application follows a browser → Flask routes → SQLite workflow. The database is accessed through parameterized SQL queries. Flask runs on the loopback interface for demonstration.

3. Identification and authentication

A user enters a username and password. The backend looks up the account in SQLite and checks the supplied password against a stored Argon2id hash. Argon2id generates a unique salt for each password. Before hashing or verification, HMAC-SHA256 combines the password with a server-side pepper kept in a private .env file, outside the database. Plaintext passwords are not stored.

4. Two-factor authentication

After successful password verification, the user must enroll or verify a six-digit time-based one-time password (TOTP) using a compatible authenticator application. Initial enrollment uses a private QR code or setup key and requires confirmation. TOTP secrets are encrypted in SQLite using a separate Fernet key from .env. Full session access is granted only after a valid code; the system records the last accepted time step to reduce code replay.

5. Role-based authorization

Inputers create and view their own sample records. Managers can create inputer accounts. Admins can delete inputer accounts but cannot create users. Administrators can create accounts with selected roles and delete permitted accounts, except the last administrator. Flask checks the current role from the database for every protected route; hiding interface buttons is not the security control.

6. Database and security controls

SQLite has users and records tables with foreign-key relationships. User passwords are stored as hashes, while encrypted TOTP secrets and last-used time steps support second-factor verification. The application uses signed HTTP-only session cookies, SameSite=Lax, form CSRF tokens, generic credential errors, basic per-process attempt throttling, request-size limits and explicit logout. Authentication and role checks occur on the server. Secrets, virtual environments and local databases are excluded from Git through .gitignore.

7. User workflow

First, the user visits the login page and enters their credentials. Second, Flask validates the password. Third, the user completes authenticator enrollment if necessary or submits a fresh TOTP code. Fourth, the system opens the dashboard and displays available actions for the current role. Managers and administrators can add accounts; admins and administrators can delete eligible accounts. Inputers can save and view their own records.

8. Testing approach and results

The included automated test suite is designed to check password hashing, encrypted TOTP storage, failed passwords, rejected incorrect 2FA codes, blocked unauthenticated access, CSRF rejection and role restrictions. Python source syntax was checked during packaging. Full runtime tests could not be executed in the packaging environment because Flask and PyOTP were not installed and package downloads were unavailable. Therefore, runtime test results are not claimed here; execute the tests locally with “python -m pytest -q” and record the actual output before submission.

9. Limitations

This is an academic local prototype, not a production deployment. The built-in Flask server does not provide HTTPS, and cookie Secure mode is disabled for local HTTP. Attempt limits are stored in process memory and reset on restart. The project does not implement full account recovery, auditing, session revocation or deployment monitoring. Bootstrap loads from a CDN unless copied locally.

10. Conclusion

The project integrates user identification, password verification, authenticator-based 2FA, offline data storage and server-side role permissions in one understandable Flask application. It provides a practical demonstration of layered access control and highlights further protections needed for real-world deployment.